

on your computer, you will most likely get different values, but the actual values do not matter to understanding the concept.

All `DimmerSwitch` objects report the same type: `class DimmerSwitch`. The extremely important takeaway is that the objects all refer to the code of the same class, which really only exists in one place. When your program starts running, Python reads through all the class definitions and remembers the locations of all the classes and their methods.

The Python Tutor website (<http://PythonTutor.com>) provides some useful tools that can help you to visualize the execution of small programs by allowing you to step through each line of your code. [Figure 3-2](#) is a screenshot from running the `DimmerSwitch` class and test code through the visualization tool, stopping execution before instantiating the first `DimmerSwitch` object.

In this screenshot, you can see that Python remembers the location of the `DimmerSwitch` class and all of its methods. While classes can contain hundreds or even thousands of lines of code, no object actually gets a copy of the class's code. Having only one copy of the code is very important, as it keeps the size of OOP programs small. When you instantiate an object, Python allocates enough memory for each object to represent its own set of the instance variables defined in the class. In general, instantiating an object from a class is memory-efficient.